CAPSTONE: SPECTRAL METHODS FOR VARIATIONAL QUANTUM LINEAR SOLVERS

ADILET AKIMSHE

Abstract. We investigate whether spectral discretization methods can reduce the number of Pauli terms required to represent differential operators in Variational Quantum Linear Solvers (VQLS). Solving linear systems of equations is fundamental to scientific computing, and VQLS offers a promising near-term quantum approach compatible with Noisy Intermediate-Scale Quantum (NISQ) devices. A key bottleneck in VQLS efficiency is the Pauli term count in the decomposition of the system matrix, which directly determines measurement overhead and circuit complexity. While finite difference discretizations yield sparse matrices, spectral methods based on Chebyshev and Legendre polynomials produce structured dense matrices whose Pauli decomposition properties are less understood in the quantum context. Through numerical experiments on one- and two-dimensional differential operators, we compare Pauli term counts, VQLS convergence, and circuit complexity across discretization strategies. Our results provide practical guidance for choosing discretization methods in quantum linear solving algorithms and contribute to identifying problem structures favorable for near-term quantum advantage.

Contents

1. Introduction 2
1.1. History 2
2. Preliminaries 2
2.1. Classical bits 2
2.2. Classical logical gates 3
2.3. Classical circuits 3
2.4. Qubits 4
2.5. Quantum gates 5
2.6. Quantum circuit 7
2.7. Bell states 8
2.8. Pauli matrices 9
2.9. VQLS 10
2.10. ODE type 11
2.11. Spectral methods 12
2.12. Simulation vs Quantum 12
3. Problem statement 13
4. Results 13
4.1. Experiment 1 14
4.2. Experiment 2 14
4.3. Experiment 3 14
5. Conclusion 14
6. Appendix 14

Date: January 2026.

References 14

1. Introduction

Quantum computers represent a paradigm shift in computational technology, leveraging the principles of quantum mechanics to process information in fundamentally different ways than classical computers. Unlike classical bits that exist in definite states of 0 or 1, quantum bits (qubits) can exist in superpositions of states, enabling quantum algorithms to explore multiple solution paths simultaneously. This property, combined with quantum entanglement and interference, allows quantum computers to potentially solve certain problems exponentially faster than their classical counterparts [1, 2].

The development of quantum algorithms for solving linear systems of equations has garnered significant attention, as linear algebra problems form the backbone of scientific computing, machine learning, and optimization. In particular, quantum spectral methods have emerged as a powerful approach for solving differential equations with complexity that scales only poly-logarithmically in the system dimension [10]. Variational Quantum Linear Solvers (VQLS) represent a promising near-term approach that is compatible with current Noisy Intermediate-Scale Quantum (NISQ) devices.

1.1. History.

The concept of quantum computing was first proposed by Richard Feynman in 1982, who suggested that quantum systems could be simulated more efficiently using quantum computers rather than classical ones [1]. David Deutsch formalized this idea in 1985 by describing a universal quantum computer. The field gained significant momentum in 1994 when Peter Shor developed his famous algorithm for integer factorization, demonstrating that quantum computers could solve certain problems exponentially faster than classical computers [1, 3].

The late 1990s and early 2000s saw the development of quantum error correction codes and the first experimental implementations of small quantum systems. Grover's search algorithm (1996) and the development of quantum simulation techniques further established the theoretical foundation of quantum computing [1].

In recent years, the focus has shifted toward NISQ devices and variational quantum algorithms. These algorithms, including the Variational Quantum Eigensolver (VQE) and VQLS, are designed to work within the constraints of current quantum hardware, which is limited by decoherence and gate errors [2, 5]. A comprehensive historical overview of quantum computing can be found in Chapter 1 of Nielsen and Chuang [1] and the introductory chapter of Hidary [2].

2. Preliminaries

This section provides the foundational concepts necessary to understand the application of spectral methods to variational quantum linear solvers.

2.1. Classical bits.

A classical bit is the fundamental unit of information in classical computing. It can exist in one of two discrete states: 0 or 1. These states typically correspond to physical quantities such as voltage levels in electronic circuits (e.g., low voltage = 0, high voltage = 1) or magnetic orientations in storage media.

Mathematically, we can represent a classical bit as an element of the set $\{0, 1\}$. Multiple bits can be combined to represent larger numbers or more complex information. For example, n classical bits can represent 2^n different states, but can only be in one specific state at any given time.

The deterministic nature of classical bits means that reading or measuring a bit does not change its state—a 0 remains 0 and a 1 remains 1 after measurement. This property is fundamentally different from quantum bits, as we will see later. In essence, a classical bit is like a coin: either heads or tails up. A rigorous treatment of classical information and computation is given in Chapter 1 of [1].

2.2. **Classical logical gates.** Classical logical gates are the building blocks of classical circuits. They perform Boolean operations on one or more input bits to produce output bits. The most common gates include:

NOT gate: Inverts the input bit. If input is 0, output is 1, and vice versa. This is the only non-trivial single-bit gate.

AND gate: Takes two inputs and outputs 1 only if both inputs are 1.

OR gate: Takes two inputs and outputs 1 if at least one input is 1.

XOR gate (Exclusive-OR): Takes two inputs and outputs 1 if the inputs are different.

NAND gate: The negation of AND; outputs 0 only if both inputs are 1.

These gates can be combined to create more complex operations. Importantly, NAND gates (and similarly NOR gates) are **universal**: any Boolean function can be implemented using only NAND gates. This property is crucial for building general-purpose computers [1, 3].

Classical gates are typically **irreversible**: given only the output, you cannot always determine the input (e.g., AND gate output of 0 could come from inputs 00, 01, or 10). This irreversibility results in an irretrievable loss of information [1].

Quantum gates, in contrast, are unitary (explained why below) and thus reversible. For a complete treatment of Boolean logic, classical gates, and their computational universality, see [1, Chapter 1].

2.3. **Classical circuits.** Classical circuits are composed of logical gates connected in specific configurations to perform computational tasks. Information flows through the circuit from input registers, through various gate operations, to output registers. Classical circuits consist of wires (to carry information) and logic gates (to manipulate the information).

A classical circuit can be represented as a directed acyclic graph (DAG), where nodes represent gates and edges represent wires carrying bit values. The depth of a circuit is the length of the longest path from input to output, which relates to the time complexity of the computation.

Classical circuits allow for operations like FANIN (joining wires together via OR operation) and FANOUT (creating multiple copies of a bit). These operations are fundamental to classical computing but, as we will see, have no direct quantum analogs.

Classical circuits are the foundation of all modern digital electronics, from simple calculators to complex processors. They implement algorithms through sequences of logical operations, with the state of the system at each step being completely determined by the previous state and the gate operations [1, 3].

Example: Half adder A canonical example of a classical circuit is the **half adder**, which adds two single-bit inputs A and B and produces a 2-bit result: a sum bit and a carry bit.

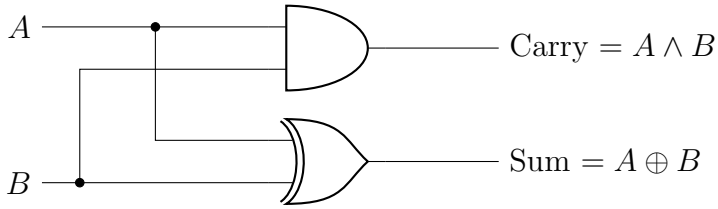


FIGURE 1. Half adder: a classical circuit computing the 2-bit sum of inputs A and B. The AND gate produces the carry bit and the XOR gate produces the sum bit. Filled dots denote wire junctions; wire crossings without a dot are not connected.

2.4. **Qubits.** A qubit (quantum bit) is the fundamental unit of quantum information. Unlike classical bits, qubits can exist in superposition states—simultaneously being in both 0 and 1 states with certain probability amplitudes.

Mathematically, a qubit state is represented as a vector in a two-dimensional complex Hilbert space:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \tag{1}$$

where α and β are complex amplitudes satisfying the normalization condition $|\alpha|^2 + |\beta|^2 = 1$ [1, 4]. The states $|0\rangle$ and $|1\rangle$ form an orthonormal basis (also called the computational basis), corresponding to the classical bit values.

When measured in the computational basis, the qubit collapses to either $|0\rangle$ with probability $|\alpha|^2$ or $|1\rangle$ with probability $|\beta|^2$. This measurement is probabilistic and irreversible—after measurement, the superposition is destroyed. For instance, a qubit in the state

$$|+\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle \tag{2}$$

yields 0 or 1 with equal probability (50% each) when measured.

Physical realizations: Qubits can be realized in various physical systems:

- Polarization states of photons
- Nuclear spin alignment in magnetic fields
- Electronic states of atoms (ground state $|0\rangle$ and excited state $|1\rangle$)
- Superconducting circuits

The Bloch Sphere: Any qubit state can be parameterized as:

$$|\psi\rangle = \cos\frac{\theta}{2}|0\rangle + e^{i\phi}\sin\frac{\theta}{2}|1\rangle \tag{3}$$

where θ and ϕ are real numbers defining a point on the unit three-dimensional sphere (see Figure 2). This representation, called the Bloch sphere, provides an excellent visualization tool for single-qubit states and operations. The north pole represents $|0\rangle$, the south pole represents $|1\rangle$, and points on the equator represent equal superpositions with different relative phases [1, 2]. A thorough introduction to qubits and the Bloch sphere is given in Section 1.2 of [1]; Chapter 3 of [2] offers a more applied perspective.

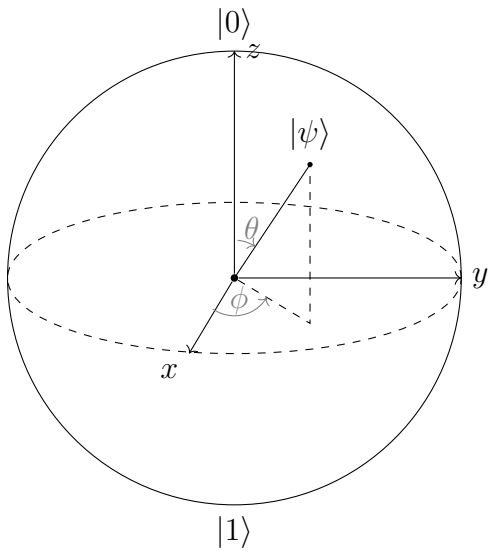


FIGURE 2. Bloch sphere representation of a qubit. The state $|\psi\rangle$ is represented as a point on the surface of a unit sphere, parameterized by angles θ and ϕ.

Information content: While the Bloch sphere has infinitely many points (suggesting infinite information storage), measurement reveals only a single classical bit. The continuous parameters α and β can only be determined from infinitely many measurements of identically prepared qubits. Nature appears to keep track of this "hidden information" during quantum evolution, and this hidden quantum information grows exponentially with the number of qubits.

2.5. **Quantum gates.** Quantum gates are unitary operations that manipulate qubit states. Unlike classical gates, quantum gates are **reversible**—given the output and the gate operation, the input can always be determined. This reversibility is expressed mathematically by the unitarity constraint: for a gate represented by matrix U, we have $U^\dagger U = I$, where $U^\dagger$ is the conjugate transpose and I is the identity matrix [1, 4].

Single-qubit gates:

The quantum gates act linearly on superposition states. For example, if we define the Pauli-X (or simply X) gate by its action on basis states, it automatically defines its action on superpositions: $X(\alpha|0\rangle + \beta|1\rangle) = \alpha|1\rangle + \beta|0\rangle$.

Pauli-X gate: The quantum analog of the NOT gate, with matrix representation:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \tag{4}$$

It flips $|0\rangle$ to $|1\rangle$ and vice versa.

Pauli-Z gate: Leaves $|0\rangle$ unchanged but flips the sign of $|1\rangle$:

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \tag{5}$$

Hadamard gate (H): Creates equal superposition and is crucial for quantum algorithms:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \tag{6}$$

It transforms: $H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = |+\rangle$ and $H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |-\rangle$. The Hadamard gate is sometimes described as a "square-root of NOT" gate. Notably, $H^2 = I$, so applying H twice returns the original state.

On the Bloch sphere, single-qubit gates correspond to rotations and reflections. The Hadamard operation is a rotation about the $\hat{y}$ axis by 90°, followed by a rotation about the $\hat{x}$ axis by 180°.

Phase gates (S, T): Apply phase shifts to the $|1\rangle$ component.

Rotation gates: $R_x(\theta)$, $R_y(\theta)$, $R_z(\theta)$ perform rotations around respective axes on the Bloch sphere by angle θ.

Two-qubit gates:

CNOT (Controlled-NOT): The prototypical multi-qubit gate, with two input qubits: a control and a target. It flips the target qubit if and only if the control qubit is $|1\rangle$. The transformation is:

$$\begin{aligned} |00\rangle &\to |00\rangle \\ |01\rangle &\to |01\rangle \\ |10\rangle &\to |11\rangle \\ |11\rangle &\to |10\rangle \end{aligned} \tag{7}$$

This can be summarized as $|A, B\rangle \to |A, B \oplus A\rangle$, where $\oplus$ is addition modulo 2 (XOR). The matrix representation is:

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \tag{8}$$

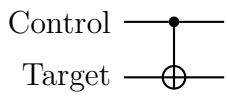


FIGURE 3. Circuit diagram for the CNOT gate. The control qubit is on top with a filled circle, and the target qubit is below with the $\oplus$ symbol.

CZ (Controlled-Z): Applies a phase flip to the target qubit if the control is $|1\rangle$.

Controlled-U gates: More generally, for any unitary U acting on n qubits, we can define a controlled-U gate with one control qubit. If the control is $|0\rangle$, nothing happens; if it's $|1\rangle$, U is applied to the target qubits.

Universality: A fundamental result in quantum computing is that **any multiple qubit logic gate may be composed from CNOT and single-qubit gates**. This is the quantum analog of the universality of the NAND gate in classical computing. In fact, we only need certain specific single-qubit rotations (not arbitrary angles) to achieve universal quantum computation—we can build arbitrarily good approximations to any quantum gate using a finite set of fixed gates [1]. Detailed proofs of universality and a complete treatment of single- and multi-qubit gates appear in Chapter 4 of [1]; Chapter 4 of [3] provides a complementary exposition.

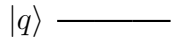
No-Cloning Theorem: Unlike classical bits, qubits cannot be copied. It is impossible to create a quantum circuit that takes an unknown quantum state $|\psi\rangle$ and produces two copies $|\psi\rangle|\psi\rangle$. This can be seen by attempting to use a CNOT gate

for copying: applying CNOT to $(\alpha|0\rangle + \beta|1\rangle)|0\rangle$ yields $\alpha|00\rangle + \beta|11\rangle$, which is not equal to $|\psi\rangle|\psi\rangle = \alpha^2|00\rangle + \alpha\beta|01\rangle + \alpha\beta|10\rangle + \beta^2|11\rangle$ unless $\alpha\beta = 0$. This fundamental limitation has profound implications for quantum information processing [1, 3].

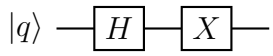
No-Deleting Theorem: Just as quantum information cannot be copied, it also cannot be destroyed. Closed quantum evolution is represented by unitary operators (which have an inverse) and therefore reversible: information about initial states is always preserved in the global system. No quantum operation can take two identical copies $|\psi\rangle|\psi\rangle$ and erase one to produce $|\psi\rangle|0\rangle$ for an arbitrary unknown state $|\psi\rangle$. This is the time-reversal dual of the no-cloning theorem, and both follow from the same underlying constraint—unitarity forbids the irreversible loss of information in an isolated quantum system.

2.6. **Quantum circuit.** A quantum circuit is a sequence of quantum gates applied to a set of qubits, analogous to classical circuits but with crucial differences [1, 7]. Quantum circuits are represented by circuit diagrams where:

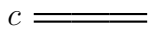
- Horizontal lines represent qubit wires (time flows left to right):



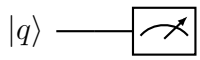
- Boxes or symbols represent quantum gates:



- Double lines represent classical bits (measurement outcomes):



- Measurement is represented by a meter symbol:



Quantum circuits must:

(1) Initialize qubits in a known state (typically $|0\rangle$)
(2) Apply a sequence of unitary quantum gates
(3) Perform measurements to extract classical information

Quantum circuits are **read from left to right**. The state of an n-qubit system is described by a vector in a 2^n-dimensional Hilbert space. After applying gates $U_1, U_2, \ldots, U_m$ to an initial state $|\psi_0\rangle$, the final state is:

$$|\psi_f\rangle = U_m \cdots U_2 U_1 |\psi_0\rangle \tag{9}$$

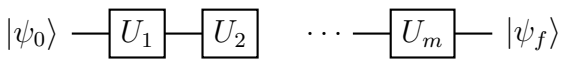


FIGURE 4. A generic quantum circuit applying gates $U_1, U_2, \ldots, U_m$ in sequence to an initial state $|\psi_0\rangle$, producing the final state $|\psi_f\rangle = U_m \cdots U_1|\psi_0\rangle$. Gates are applied left to right, but compose right to left in matrix notation.

Differences from classical circuits:

- Quantum circuits are **acyclic**—no feedback loops are allowed
- No FANIN operation (joining wires), as this is not reversible
- No FANOUT operation (copying qubits), due to the no-cloning theorem
- All operations except measurement are reversible (unitary)

Example: SWAP circuit A useful quantum circuit swaps the states of two qubits using three CNOT gates:

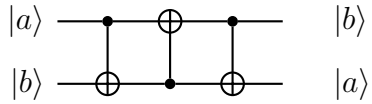


FIGURE 5. SWAP circuit using three CNOT gates to exchange the states of two qubits.

The operation can be verified: $|a, b\rangle \to |a, a \oplus b\rangle \to |a \oplus (a \oplus b), a \oplus b\rangle = |b, a \oplus b\rangle \to |b, (a \oplus b) \oplus b\rangle = |b, a\rangle$.

Multiple Qubits: For n qubits, the computational basis states are of the form $|x_1 x_2 \ldots x_n\rangle$, and a quantum state is specified by 2^n complex amplitudes. For $n = 500$, this number exceeds the estimated number of atoms in the Universe! This exponential scaling represents enormous potential computational power, as Nature appears to manipulate such quantities during quantum evolution.

2.7. **Bell states.** Bell states (also called EPR pairs, after Einstein, Podolsky, and Rosen) are maximally entangled two-qubit states that exhibit remarkable quantum correlations [1]. The four Bell states are:

$$|\beta_{00}\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}} \tag{10}$$

$$|\beta_{01}\rangle = \frac{|01\rangle + |10\rangle}{\sqrt{2}} \tag{11}$$

$$|\beta_{10}\rangle = \frac{|00\rangle - |11\rangle}{\sqrt{2}} \tag{12}$$

$$|\beta_{11}\rangle = \frac{|01\rangle - |10\rangle}{\sqrt{2}} \tag{13}$$

These states can be generated using a simple quantum circuit consisting of a Hadamard gate followed by a CNOT gate:

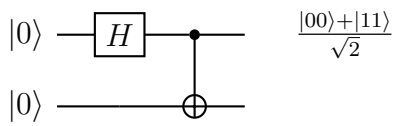


FIGURE 6. Circuit for generating the Bell state $|\beta_{00}\rangle$. The Hadamard gate creates superposition, and the CNOT entangles the qubits.

Properties and significance:

- **Entanglement:** Measuring one qubit instantly determines the measurement outcome of the other, regardless of spatial separation
- **Non-classical correlations:** John Bell proved that measurement correlations in Bell states are stronger than could ever exist between classical systems
- **Quantum teleportation:** Bell states are the key resource for quantum teleportation, allowing quantum information to be transmitted using only classical communication and pre-shared entanglement

- **Superdense coding:** Bell states enable sending two classical bits using only one qubit
- **Quantum cryptography:** Used in quantum key distribution protocols

When measuring the first qubit of $|\beta_{00}\rangle$, we obtain:

- Result 0 with probability 1/2, leaving post-measurement state $|00\rangle$
- Result 1 with probability 1/2, leaving post-measurement state $|11\rangle$

Thus, measurement outcomes are perfectly correlated—if the first qubit measures 0, the second must also measure 0, and similarly for 1. These correlations persist for other types of measurements and cannot be explained by classical hidden variables. For a comprehensive treatment of quantum entanglement, the EPR argument, and Bell inequalities, we refer to Chapter 2 of [1] and Chapter 7 of [3].

2.8. **Pauli matrices.** The Pauli matrices are a set of three 2×2 complex Hermitian matrices that form a basis for the space of 2×2 Hermitian matrices (along with the identity matrix). They are fundamental operators in quantum mechanics and quantum computing [1, 4].

The three Pauli matrices are:

$$\sigma_x = X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad \sigma_y = Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad \sigma_z = Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \tag{14}$$

Key properties:

- They are Hermitian: $\sigma_i^\dagger = \sigma_i$
- They are unitary: $\sigma_i^2 = I$ (the identity matrix)
- They satisfy anticommutation relations: $\{\sigma_i, \sigma_j\} = 2\delta_{ij} I$
- They satisfy commutation relations: $[\sigma_i, \sigma_j] = 2i\epsilon_{ijk}\sigma_k$
- They are traceless: $\mathrm{Tr}(\sigma_i) = 0$

Pauli strings for multi-qubit systems: For multi-qubit systems, Pauli strings are formed by tensor products of single-qubit Pauli operators. For example, for a three-qubit system: $X \otimes Z \otimes I$, which applies X to the first qubit, Z to the second, and identity (nothing) to the third.

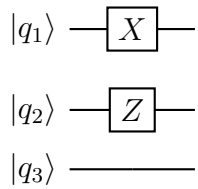


FIGURE 7. Circuit representation of the Pauli string $X \otimes Z \otimes I$. An X gate acts on qubit 1, a Z gate acts on qubit 2, and qubit 3 passes through unchanged (identity).

These Pauli strings form a complete basis for decomposing arbitrary operators on n qubits. Any $2^n \times 2^n$ matrix can be written as a linear combination of 4^n Pauli strings (including identity). This decomposition is crucial for quantum algorithms.

Relevance to VQLS: In the context of VQLS, the number of Pauli terms in the decomposition of the system matrix directly impacts:

- Circuit complexity (more terms require more measurements)
- Measurement overhead (each Pauli string requires separate measurement circuits)

- Overall algorithm efficiency

Reducing the number of Pauli terms is therefore a key optimization target for VQLS implementations. The algebraic properties of the Pauli matrices and their use as an operator basis are developed in detail in Chapter 1 of [1] and Chapter 3 of [4].

2.9. **VQLS.** Variational Quantum Linear Solver (VQLS) is a quantum algorithm designed to solve linear systems of equations $A|\mathbf{x}\rangle = |\mathbf{b}\rangle$ on near-term quantum devices [9]. Unlike the HHL algorithm, VQLS is a hybrid quantum-classical algorithm suitable for NISQ devices.

Analogy with machine learning. VQLS is structurally very similar to supervised machine learning. In both cases:

- A *model with trainable parameters* $\boldsymbol{\theta}$ is defined (a neural network in ML; a parameterized quantum circuit in VQLS).
- A *loss function* measures how far the current parameters are from the desired output.
- A *classical optimizer* (e.g. gradient descent) iteratively updates $\boldsymbol{\theta}$ to minimize the loss.
- Training *converges* when the loss is sufficiently small.

The key difference is that in VQLS the model lives on a quantum computer: the "forward pass" is a quantum circuit measurement, and the expressivity of the model comes from quantum superposition and entanglement rather than from the depth of a neural network [5, 6].

The VQLS algorithm works by:

1. **Ansatz preparation:** Preparing a parameterized quantum state $|\psi(\boldsymbol{\theta})\rangle = V(\boldsymbol{\theta})\,|0\rangle$ using a variational quantum circuit. A common choice is the hardware-efficient ansatz (see Figure 8).

Ansatz analogy. The word *ansatz* (German for "initial guess" or "approach") refers to a family of quantum states parameterized by angles $\boldsymbol{\theta}$. Think of it like a function template: just as a data scientist might choose a neural network architecture (how many layers, what activations) before training, a quantum engineer chooses an ansatz circuit structure. The parameters $\boldsymbol{\theta}$ are then adjusted during training to find the best approximation to the solution of the linear system. The choice of ansatz is crucial: it must be expressive enough to capture the solution but also efficient enough to run on NISQ hardware.

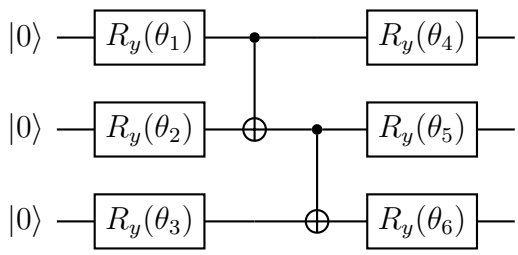


FIGURE 8. Example of a 3-qubit Hardware-Efficient Ansatz. R_y gates provide trainable rotations, while CNOTs provide entanglement.

2. **Cost function:** Defining a cost function that measures how well $A\,|\psi(\boldsymbol{\theta})\rangle$ aligns with $|b\rangle$. This often involves the Hadamard Test (Figure 9) to compute the terms $\langle b|A|\psi(\boldsymbol{\theta})\rangle$.

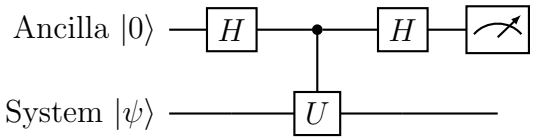


FIGURE 9. The Hadamard Test circuit used to evaluate overlaps. In VQLS, U is replaced by operations related to A and the state preparation of b.

3. **Classical optimization:** Using a classical optimizer (like gradient descent) to adjust $\boldsymbol{\theta}$ to minimize $C(\boldsymbol{\theta})$.

4. **Measurement:** Evaluating expectation values through quantum measurements.

Pauli decomposition and efficiency: The computational cost of VQLS depends heavily on how the matrix A is decomposed into Pauli strings:

$$A = \sum_{i=1}^{M} c_i P_i \tag{15}$$

Each Pauli term requires separate measurement circuits, so minimizing M is crucial for efficiency. This is where discretization methods become important.

2.10. **ODE type.** Ordinary Differential Equations (ODEs) are equations involving functions and their derivatives. Many scientific and engineering problems can be formulated as ODEs [4, 8]. A general form of an ODE is:

$$\frac{dy}{dt} = f(t, y) \tag{16}$$

where y is the unknown function and f is a known function.

Linear ODEs are particularly important and can be written as:

$$\frac{dy}{dt} + p(t)y = q(t) \tag{17}$$

When discretized, ODEs can be converted into linear systems of equations. Different discretization methods (finite difference, finite element, spectral methods) lead to different matrix structures. For quantum linear solvers, the choice of discretization method affects:

- The sparsity pattern of the resulting matrix
- The condition number of the system
- The number of Pauli terms needed for quantum representation
- The overall quantum circuit depth and complexity

Example - Second-order ODE (Poisson equation): Consider the boundary value problem:

$$-\frac{d^2u}{dx^2} = f(x), \quad u(0) = u(1) = 0 \tag{18}$$

This is the one-dimensional Poisson equation, a canonical PDE benchmark in quantum computing research [11]. Finite difference discretization with n grid points yields a tridiagonal matrix, while spectral methods using Chebyshev or Legendre polynomials produce dense but structured matrices [10]. Understanding which approach leads to fewer Pauli terms is the central question of this research.

2.11. **Spectral methods.** Spectral methods are a class of numerical techniques for solving differential equations that represent the solution as a sum of basis functions (typically orthogonal polynomials or Fourier modes). Unlike finite difference or finite element methods that provide pointwise approximations, spectral methods approximate the global behavior of the solution [4].

Common spectral bases include:

- **Fourier series:** For periodic problems on regular domains

$$u(x) = \sum_{k=-N}^{N} \hat{u}_k e^{ikx} \tag{19}$$

- **Chebyshev polynomials:** For non-periodic problems on bounded intervals $[-1, 1]$

$$u(x) = \sum_{k=0}^{N} \hat{u}_k T_k(x) \tag{20}$$

  where $T_k(x) = \cos(k \arccos(x))$
- **Legendre polynomials:** For problems on $[-1, 1]$ with natural boundary conditions

$$u(x) = \sum_{k=0}^{N} \hat{u}_k P_k(x) \tag{21}$$

The key advantages of spectral methods are:

- **Exponential convergence** for smooth solutions (spectral accuracy)
- High accuracy with fewer degrees of freedom compared to finite differences
- Dense but structured matrices with special mathematical properties
- Well-understood conditioning and stability properties

Disadvantages:

- Dense matrices (though structured)
- Less flexible for complex geometries
- Reduced accuracy for non-smooth solutions

Connection to VQLS: In the context of VQLS, spectral methods produce matrices with specific structures that may decompose into fewer Pauli terms compared to standard finite difference discretizations. The dense but structured nature of spectral matrices could potentially be exploited for more efficient Pauli decompositions, reducing quantum circuit complexity. This hypothesis is motivated by the work of Childs and Liu [10], who showed that spectral discretizations yield quantum algorithms for ODEs with complexity $\text{poly}(\log d, \log(1/\epsilon))$ — exponentially better in the error ϵ than finite-difference-based approaches. This hypothesis forms the core of our research question.

2.12. **Simulation vs Quantum.** Classical simulation of quantum circuits is possible but becomes exponentially expensive as the number of qubits increases. A classical computer must track 2^n complex amplitudes for an n-qubit system, making exact simulation intractable for systems with more than approximately 50 qubits [1, 2].

However, for small-scale problems, classical simulation is valuable for:

- Algorithm development and testing
- Verification of quantum results

- Benchmarking quantum advantage
- Understanding noise effects
- Debugging quantum circuits before running on expensive hardware

The transition from simulation to actual quantum hardware introduces challenges:

- **Noise and errors:** Real quantum devices suffer from decoherence and gate errors
- **Limited connectivity:** Qubits may not all be directly connected, requiring SWAP gates
- **Measurement limitations:** Statistical sampling is needed to estimate expectation values
- **Calibration:** Gate fidelities vary and require regular calibration
- **Coherence time:** Limited time before quantum information is lost

For this project, classical simulation allows us to investigate whether spectral methods reduce Pauli term counts in a controlled environment before considering implementation on actual quantum hardware. The simulation results will inform whether the approach is promising enough to warrant the complexity and cost of real quantum execution.

3. Problem statement

Standard discretization methods for differential equations, such as finite difference schemes, often produce sparse matrices when converted to quantum representations. However, these matrices may decompose into a large number of Pauli terms, which increases the measurement overhead and circuit complexity for variational quantum linear solvers.

Spectral methods offer an alternative discretization approach that produces different matrix structures—typically denser but with specific mathematical properties. The central question of this research is: **Can spectral methods reduce the number of Pauli terms needed to represent discretized differential operators in VQLS, thereby improving the efficiency of quantum linear solving algorithms?**

More specifically, we investigate:

(1) How does the Pauli term count compare between spectral methods and finite difference methods for common differential operators?
(2) Does the structured nature of spectral method matrices lead to more efficient Pauli decompositions?
(3) What is the trade-off between matrix density and Pauli term count?
(4) How does this affect the overall quantum circuit depth and measurement requirements for VQLS?
(5) Does the exponential convergence of spectral methods provide any advantage in the quantum setting?

Understanding this relationship could lead to significant improvements in VQLS implementations for solving differential equations, which have applications across scientific computing, fluid dynamics, quantum chemistry, and machine learning.

4. Results

This section presents the experimental results comparing Pauli term counts and VQLS performance for spectral methods versus traditional finite difference discretizations.

4.1. **Experiment 1.** 1D figures

4.2. **Experiment 2.** 2D figures

4.3. **Experiment 3.** f not polynomial. Approximation error? figures

5. Conclusion

[Summary of findings regarding Pauli term reduction with spectral methods, implications for VQLS algorithm design, practical recommendations for choosing discretization methods in quantum linear solvers, and future work directions including testing on real quantum hardware]

6. Appendix

Code
More pictures

References

[1] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th Anniversary Edition, Cambridge University Press, 2011.
[2] J. D. Hidary, *Quantum Computing: An Applied Approach*, 2nd ed., Springer, 2021.
[3] N. S. Yanofsky and M. A. Mannucci, *Quantum Computing for Computer Scientists*, Cambridge University Press, 2008.
[4] W. Scherer, *Mathematics of Quantum Computing: An Introduction*, Springer, 2019.
[5] M. Schuld and F. Petruccione, *Supervised Learning with Quantum Computers*, Springer, 2018.
[6] P. Wittek, *Quantum Machine Learning: What Quantum Computing Means to Data Science*, Academic Press, 2014.
[7] E. Johnston, N. Harrigan, and M. Gimeno-Segovia, *Programming Quantum Computers: Essential Algorithms and Code Samples*, O'Reilly Media, 2019.
[8] A. Woody, *Essential Mathematics for Quantum Computing*, Packt Publishing, 2022.
[9] C. Bravo-Prieto, R. LaRose, M. Cerezo, Y. Subasi, L. Cincio, and P. J. Coles, *Variational Quantum Linear Solver*, Quantum **7**, 1188 (2023). arXiv:1909.05820.
[10] A. M. Childs and J.-P. Liu, *Quantum Spectral Methods for Differential Equations*, Communications in Mathematical Physics **375**, 1427–1457 (2020). arXiv:1901.00961.
[11] A. Mukhanbet, N. Azatbekuly, and B. Daribayev, *Optimizing Quantum Algorithms for Solving the Poisson Equation*, Scientific Journal of Astana IT University, vol. 18, pp. 55–65, June 2024. doi:10.37943/18REAT9767.